ARC_SpectraPro.dll

The ARC_Spectrapro.dll is provided by Acton Research Corp (ARC) to allow for simplified low-level control of up to 16 Acton Research Corp. SpectraPro/AM/VM Monochromators and Spectrographs. At this time the DLL does not work with ARC NCL and external ARC Filter controllers.

The Dll is broken into groups of functions.

1. Communications : These functions are used for setting up communications with the ARC instrument(s) attached.
2. Information : These functions provided basic information about Open ARC instruments. This information includes such things as; model, serial number, focal length, half angle, detector angle, and if the instrument is a double monochromator.
3. Wavelength : These functions set and read the current wavelength (center wavelength for a Spectrograph) of an instrument. Functions are provided for the most common wavelength units, but it should be noted that all ARC instruments internally use nanometers (nm), so all functions call ARC_get_Mono_Wavelength_nm / ARC_set_Mono_Wavelength_nm and then convert to the requested units.
4. Grating : These functions are used for selecting the current grating and providing information about the grating.
5. Diverter Mirrors : These functions provide control over diverter mirror in instruments with multiple entrance or exit ports. They are not valid for instruments not equipped with these options. Note these functions do not work with all older SD-748/SP-275/SP-500(pre SP-500i)/SP-750 (pre SP-750i) controllers.
6. Slits : These functions provide control over motorized slits (both continuously variable and index able). They are not valid for instruments not equipped with these options. Note these functions do not work with all older SD-748/SP-275/SP-500(pre SP-500i)/SP-750 (pre SP-750i) controllers.
7. Filters : These functions provide control of filter wheels in instruments that contain and integrated filter wheel controller. At present this is not a standard feature and the functions are invalid for most instruments. This function does not provide control of a filter wheel controller external to the monochromator.
8. Advanced Functions : These are advanced functions that most normal users of the DLL will not need. They allow for debugging of hardware and installing/adjusting gratings.
9. Functions for the controlling External Filter Wheel Controllers.
    1. Communications : These functions are used for setting up communications with the ARC instrument(s) attached.
    2. Information : These functions provided basic information about Open ARC instruments. This information includes such things as; model and serial number. (Note, older filter controllers do not store the serial number electronically).
    3. Filters : These functions provide control of the external ARC Filter controllers.

Working with the DLL is a multi step process.

1. First identify the number of monochromators / spectrographs attached by calling ARC_Search_For_Mono. Each instruments enumerates starting with zero up to the number of instruments found minus one. ARC_get_Mono_preOpen_Model will return the model string in a variant for the enumerated value passed to it.
2. ARC_Open_Mono is then called to open the enumeration. The function will return false (result = 0) if it failed to open the enumeration.
3. The user the calls the functions he needs to use to operate the instrument. Once opened, a monochromator port can be left open until the software exits.
4. When the user is done with the instrument (usually program exit) the user calls ARC_Close_Mono to close the port.

Data types used :
Most functions return a 16 bit unsigned integer result. True is defined as not equal to zero. The function failed if true is not returned.

In VB this is an integer
In VCC this is an integer
In Delphi this is a WordBool

Parameters passed to and from a function are of the following types …

Integers values are 32 bit integers
In VB this is a long
In VCC this is a long
In Delphi this is an integer

Floating-point values are doubles. This is true for most languages.

Strings are wrapped inside of the variant data type. This allows compatibility with most programming languages
In VB this is a variant
In VCC this is a variant
In Delphi the is an OLEVaraint (OLEVariant does not work the same as variant, you must include variants in the uses clause on Delphi 6 and higher)

Boolean values are 16 bit integers defined as true if not equal to zero.
In VB this is an integer
In VCC this is an integer
In Delphi this is a wordbool

Functions Provided :
Communications – Monochromators are opened in one of two ways, using ARC_Search_For_Mono/ARC_Open_Mono or to directly open the COM port with ARC_Open_Mono_Port.

ARC_Search_For_Mono , function used to search for attached monochromators. With the exception of ARC_Ver, must be the first function called.

ARC_get_Num_Found_Mono_Ports , Returns the value last returned by **ARC_Search_For_Mono**. Can be called multiple times.

ARC_Open_Mono , function to open a monochromator

ARC_Open_Mono_Port , function to open a monochromator by addressing a specific COM port.

ARC_Close_Mono , function to close an open monochromator

ARC_Valid_Mono_Enum , function returns is a monochromator enumeration is both valid and the instrument is open.

ARC_get_Mono_preOpen_Model , returns the model string of an instrument not yet opened. Note, ARC_Search_For_Mono needs to be called prior to this call.

direct communications function

ARC_Send_CMD_To_Mono , this is an internal function that provides direct communications with the instrument. Usage of this function should be avoided.

ARC_Ver , this function provides the DLL version. It is the only function that can be called at any time.

Information

ARC_get_Mono_Model , returns the model string from the instrument

ARC_get_Mono_Serial , returns the serial number from the instrument

ARC_get_Mono_Focallength , returns the default focal length of the instrument

ARC_get_Mono_HalfAngle , returns the default half angle of the instrument in radians

ARC_get_Mono_DetectorAngle , returns the default detector angle of the instrument in radians

ARC_get_Mono_Double , returns if the instrument is a double monochromator

ARC_get_Mono_Precision , returns the nm decimal precision of the wavelength drive. Note this is independent of the wavelength step resolution whose coarseness is defined by the grating being used.

Wavelength

ARC_get_Mono_Wavelength_nm , returns the current center wavelength of instrument in nm.

ARC_set_Mono_Wavelength_nm , sets the center wavelength of the instrument in nm.

ARC_get_Mono_Wavelength_ang , returns the current center wavelength of instrument in Angstroms. Converted from nm.

ARC_set_Mono_Wavelength_ang , sets the center wavelength of the instrument in angstroms. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_eV , returns the current center wavelength of instrument in electron Volts. Converted from nm.

ARC_set_Mono_Wavelength_eV , sets the center wavelength of the instrument in electron Volts. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_micron , returns the current center wavelength of instrument in microns. Converted from nm.

ARC_set_Mono_Wavelength_micron , sets the center wavelength of the instrument in microns. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_absCM , returns the current center wavelength of instrument in absolute wave number. Converted from nm.

ARC_set_Mono_Wavelength_absCM , sets the center wavelength of the instrument in absolute wavenumbers. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_relCM , returns the current center wavelength of instrument in relative wavenumber based upon the center line value passed to the function in nm. Converted from nm.

ARC_set_Mono_Wavelength_relCM , sets the center wavelength of the instrument in relative wave numbers centered on the wavelength passed in nm. Converted to nm and then sent the instrument.

ARC_get_Mono_Wavelength_Cutoff_nm , returns in nm the max wavelength achievable by the instrument using the current grating. Note, value might be higher than physically achievable on linear drives using the older SD-748 based controllers.

ARC_get_Mono_Wavelength_Min_nm , returns in nm the min wavelength achievable by the instrument using the current grating. Note, value might be lower than physically achievable on linear drives using the older SD-748 based controllers.

Grating

ARC_get_Mono_Turret , returns the current grating turret

ARC_set_Mono_Turret , sets the current grating turret

ARC_get_Mono_Turret_Gratings , returns the number of gratings per turret

ARC_get_Mono_Turret_Max , return the number of turrets installed

ARC_get_Mono_Grating , returns the current grating

ARC_set_Mono_Grating , sets the current grating

ARC_get_Mono_Grating_Blaze , returns the blaze of a given grating

ARC_get_Mono_Grating_Density , returns the groove density (grooves per millimeter) of a given grating

ARC_get_Mono_Grating_Installed , returns if a grating is installed

ARC_get_Mono_Grating_Max , returns the max grating position installed (this is usually the number of gratings installed)

Mirrors

Mirror Position 1 … motorized entrance diverter mirror
Mirror Position 2 … motorized exit diverter mirror
Mirror Position 3 … motorized entrance diverter on a double slave unit
Mirror Position 4 … motorized exit diverter on a double slave unit
Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit
ARC_get_Mono_Diverter_Valid , returns if a motorized diverter position is valid for an instrument
ARC_get_Mono_Diverter_Pos , returns the slit the mirror is pointing to
ARC_set_Mono_Diverter_Pos , sets the port that the unit is to point to
ARC_get_Mono_Diverter_Pos_Var , returns a string describing the port the mirror is pointing to

Slits

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit
Slit Type 0 … No slit
Slit Type 1 … Manual Slit
Slit Type 2 … Fixxed Slit Width
Slit Type 3 … Focal Plane adapter
Slit Type 4 … Continuous Motor
Slit Type 5 … Fiber Optic
Slit Type 6 … Index able Slit
Slit Type 9 … Special Continuous Motor
ARC_get_Mono_Slit_Type , returns the slit type for a given slit
ARC_get_Mono_Slit_Type_Var , returns a string descriptor of a given slit
ARC_get_Mono_Slit_Width , returns the slit width of a motorized slit (Types 4,6, and 9)
ARC_set_Mono_Slit_Width , sets the slit width of a motorized slit (Types 4,6, and 9)
ARC_Mono_Slit_Home , homes a motorized slit (Types 4,6, and 9)
ARC_Mono_Slit_Name_Var , returns a text descriptor of a slit position
ARC_Calc_Mono_Slit_BandPass , calculates and returns the band pass for a provided slit width
ARC_get_Mono_Slit_BandPass , returns the band pass of a motorized slit (Types 4,6, and 9)
ARC_set_Mono_Slit_BandPass , sets a motorized slit width of a motorized slit (Types 4,6, and 9)

Filter

ARC_get_Mono_Filter_Present , returns if the instrument has an integrated filter wheel
ARC_get_Mono_Filter_Position , returns the current filter position
ARC_set_Mono_Filter_Position , sets the current filter position
ARC_get_Mono_Filter_Min_Pos , returns the min. filter position
ARC_get_Mono_Filter_Max_Pos , returns the max. filter position
ARC_Mono_Filter_Home , homes the filter wheel

Advanced Functions

Gear

ARC_get_Mono_Int_Led_On , returns if the interrupter LED is on
ARC_set_Mono_Int_Led , turns on and off the interrupter LED
ARC_get_Mono_Motor_Int , read the motor gear interrupter
ARC_get_Mono_Wheel_Int , read the wheel gear interrupter
ARC_Mono_Move_Steps , move the grating a set number of steps

Grating Values

ARC_get_Mono_Init_Grating , returns the initial grating (on instrument reboot)
ARC_set_Mono_Init_Grating , sets the initial grating (on instrument reboot)
ARC_get_Mono_Init_Wave_nm , returns the initial wavelength (on instrument reboot)
ARC_set_Mono_Init_Wave_nm , sets the initial wavelength (on instrument reboot)
ARC_get_Mono_Init_ScanRate_nm , returns the initial wavelength scan rate (on instrument reboot)
ARC_set_Mono_Init_ScanRate_nm , sets the initial wavelength scan rate (on instrument reboot)
ARC_get_Mono_Grating_Offset , returns the offset if a grating
ARC_get_Mono_Grating_Gadjust , returns the GAdjust of a grating
ARC_set_Mono_Grating_Offset , sets the grating offset
ARC_set_Mono_Grating_Gadjust , sets the grating GAdjust
ARC_Mono_Grating_Calc_Offset , calculate a new grating offset
ARC_Mono_Grating_Calc_Gadjust , calculate a new grating GAdjust
ARC_Mono_Grating_UnInstall , Uninstall a grating
ARC_Mono_Grating_Install , Install a grating
ARC_Mono_Reset , Reboot an instrument
ARC_Mono_Restore_Factory_Settings , restore the instrument factory settings
ARC_Mono_Save_Factory_Settings, reserved

Scan Values

ARC_get_Mono_Scan_Rate_nm_min , return the current wavelength scan rate
ARC_set_Mono_Scan_Rate_nm_min , set the wavelength scan rate
ARC_Mono_Start_Scan_To_nm , start a wavelength scan
ARC_Mono_Scan_Done , check if a scan has completed
ARC_Mono_Start_Jog , start jogging the wavelength of an instrument
ARC_Mono_Stop_Jog , end a wavelength jog

External Filter Communications – Filter Wheels are opened in one of two ways, using ARC_Search_For_Mono/ARC_Open_Filter or to directly open the COM port with ARC_Open_Filter_Port.

ARC_Open_Filter , function to open a filter controller.
ARC_Open_Filter _Port , function to open a filter controller by addressing a specific COM port.
ARC_Valid_ Filter _Enum , function returns if a filter controller enumeration is both valid and the instrument is open.
ARC_get_Filter_preOpen_Model , returns the model string of an instrument not yet opened. Note, ARC_Search_For_Filter needs to be called prior to this call.

direct communications function

ARC_Send_CMD_To_Filter , this is an internal function that provides direct communications with the instrument. Usage of this function should be avoided.

External Filter Information

ARC_get_Filter_Model , returns the model string from the instrument
ARC_get_Filter_Serial , returns the serial number from the instrument

Filter

ARC_get_Filter_Present , returns if the instrument has an integrated filter wheel
ARC_get_Filter_Position , returns the current filter position
ARC_set_Filter_Position , sets the current filter position
ARC_get_Filter_Min_Pos , returns the min. filter position
ARC_get_Filter_Max_Pos , returns the max. filter position
ARC_Filter_Home , homes the filter wheel

Pre Open Model Types as Integers

Model Type –1 ... Unknown, not supported
Model Type 0 ... Unknown, not supported
Model Type 1 ... GIMS (reserved for, none currently supported by dll)
Model Type 2 ... SP-150 (superceded by SP-2-150 (SPv6x) in 2003)
Model Type 3 ... SPx00i (generic gen2 board) (superceded by SPv6x in 2003)
Model Type 4 ... SPx004i (generic SD-2) (superceded by SD-3 in 2003)
Model Type 5 ... SP-300i (superceded by SP-2-300i (SPv6x) in 2003)
Model Type 6 ... SP-500i (superceded by SP-2-500i (SPv6x) in 2003)
Model Type 7 ... SP-750i (superceded by SP-2-750i (SPv6x) in 2003)
Model Type 8 ... INS-300 (serial number is < 42) (superceded by SPv6x based INS-300 in 2003)
Model Type 9 ... VM-502i (VM-502 with SD-2) (superceded by SD-3 based VM-502 in 2003)
Model Type 10 ... VM-504i (VM-504 with SD-2) (superceded by SD-3 based VM-504 in 2003)
Model Type 11 ... VM-503 (VM-503 with SD-2) (superceded by SD-3 based VM-503 in 2003)
Model Type 12 ... VM-505 (VM-505 with SD-2) (superceded by SD-3 based VM-505 in 2003)
Model Type 13 ... VM-506 (VM-506 with SD-2) (superceded by SD-3 based VM-506 in 2003)
Model Type 14 ... VM-507 (VM-507 with SD-2) (superceded by SD-3 based VM-507 in 2003)
Model Type 15 ... VM-510 (VM-510 with SD-2) (superceded by SD-3 based VM-510 in 2003)
Model Type 16 ... VM-511 (VM-511 with SD-2) (superceded by SD-3 based VM-511 in 2003)
Model Type 17 ... VM-512 (VM-512 with SD-2) (superceded by SD-3 based VM-512 in 2003)
Model Type 18 ... VM-521 (VM-521 with SD-2) (superceded by SD-3 based VM-521 in 2003)
Model Type 19 ... VM-523 (VM-523 with SD-2) (superceded by SD-3 based VM-523 in 2003)
Model Type 20 ... SP-275 (superceded by SP-300i in 1996)
Model Type 21 ... SP-500 (SP-500 with SD-748) (superceded by SP-500i in 1996)
Model Type 22 ... SP-750 (SP-750 with SD-748) (superceded by SP-750i in 1998)
Model Type 23 ... VM-502 (VM-502 with SD-748) (superceded by VM-502i in 1999)
Model Type 24 ... VM-504 (VM-504 with SD-748) (superceded by VM-504i in 1999)
Model Type 25 ... SD-748 (superceded by SD-2 in 1999)
Model Type 26 ... INS-150
Model Type 27 ... SPv6x (All SpectraPro produced since 2003,
Includes: SP-2-150, SP-2-300i, SP-2-500i, SP-2-750i, SD-3,
InSight, MS-2-150i, MS-2-300i, INS-300 serial numbers > 42)
Model Type 28 ... NCL (superceded by SpectraHub in 2003)
Model Type 29 ... Filter Controller
Model Type 30 ... Reserved for Sample Chambers
Model Type 31 ... Reserved (custom, not supported by dll)
Model Type 32 ... Reserved (custom, not supported by dll)
Model Type 33 ... SpectraHub

ARC_get_Mono_preOpen_Model_int32 , returns the model type of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call.

ARC_get_Filter_preOpen_Model_int32, returns the model type of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call.

Pre Open Serial Number Integers

ARC_get_Filter_preOpen_Serial_int32 , returns the serial number (as an integer) of a filterwheel not yet open.

ARC_get_Mono_preOpen_Serial_int32 , returns the serial number (as an integer) of a monochromator not yet open.

ARC_Search_For_Mono , function used to search for attached monochromators. With the exception of ARC_Ver, must be the first function called.

Delphi : function ARC_Search_For_Mono(out Num_Found : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Search_For_Mono Lib "ARC_SpectraPro.dll" (Num_Found As Long) As Integer

Num_Found : The number of Acton Research Instruments found and enumerated. Enumeration list starts with zero and ends with Num_Found minus one.

Result : True, if any instruments where found.

ARC_get_Num_Found_Mono_Ports , Returns the value last returned by **ARC_Search_For_Mono**. Can be called multiple times.

Delphi : function ARC_get_Num_Found_Mono_Ports : integer; stdcall;
VB : Private Declare Function ARC_get_Num_Found_Mono_Ports Lib "ARC_SpectraPro.dll" As Long

Result : Returns the value last returned by **ARC_Search_For_Mono**.

ARC_Open_Mono , function to open a monochromator

Delphi : function ARC_Open_Mono(Enum_Num : integer; out Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Open_Mono Lib "ARC_SpectraPro.dll" (ByVal Enum_Num As Long, Mono_Enum As Long) As Integer

Enum_Num : Number in the enumeration list to open.
Mono_Enum : The handle by which to address the monochromator in all future calls. Can be validated with ARC_Valid_Mono_Enum.
Result : True, if it successfully opened the device.

ARC_Open_Mono_Port , function to open a monochromator on a specific COM Port (ie. COM1, COM2, ect…)

Delphi : function ARC_Open_Mono_Port(Com_Num : integer; out Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Open_Mono_Port Lib "ARC_SpectraPro.dll" (ByVal Com_Num As Long, Mono_Enum As Long) As Integer

Com_Num : Com Port (COM1,COM2,ect.) to open.
Mono_Enum : The handle by which to address the monochromator in all future calls. Can be validated with ARC_Valid_Mono_Enum.
Result : True, if it successfully opened the device.

ARC_Close_Mono , function to close an open monochromator

Delphi : function ARC_Close_Mono(Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Close_Mono Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True, if the operations was carried out successfully.

ARC_Valid_Mono_Enum , function returns if a monochromator enumeration is both valid and the instrument is open. All functions call this function to verify that the requested action is valid.

Delphi : function ARC_Valid_Mono_Enum(Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Valid_Mono_Enum Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if Mono_Enum is a valid Monochromator that is open.

ARC_get_Mono_preOpen_Model , returns the model string of an instrument not yet opened. Note, ARC_Search_For_Mono needs to be called prior to this call. In the case of multiple Monochromator/Spectrographs attached, it allows the user to sort, which instruments are to be opened before opening them.

Delphi : function ARC_get_Mono_preOpen_Model(Enum_Num : integer; out Model_Var : olevariant) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_preOpen_Model Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Model_Var As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Model_Var : The model string of the unopened monochromator.
Result : True if the operation was successful

ARC_Send_CMD_To_Mono , this is an internal function that provides direct communications with the instrument. Usage of this function should be avoided.

Delphi : function ARC_Send_CMD_To_Mono(Mono_Enum : integer; SendCMD : olevariant; out RCV_Var : olevariant; ms_Timeout : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_Send_CMD_To_Mono Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal SendCMD As Variant, RCV_Var As Variant, ByVal ms_Timeout As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
SendCMD : The text command to be sent to the instrument. Note, line termination is handled by the DLL.
RCV_Var : The reply from the instrument to the given command.
Ms_Timeout : How long the DLL should wait in milliseconds for an acknowledgement before returning
Result : True if the instrument did not reject the command or timeout.

ARC_Ver , this function provides the DLL version. It is the only function that can be called at any time.

Delphi : function ARC_Ver(out Major : integer; out Minor : integer; out Build : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_Ver Lib "ARC_SpectraPro.dll" (Majorval As Long, Minorval As Long, Buildval As Long) As Integer

Major : Version Major Number
Minor : Version Minor Number
Build : Version Build Number
Result : True if the operation was successful

ARC_get_Mono_Model , returns the model string from the instrument

Delphi : function ARC_get_Mono_Model(Mono_Enum : integer;out Model_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Model Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Model_str As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Model_Var : Returns the model string of the instrument.
Result : True if the operation was successful

ARC_get_Mono_Serial , returns the serial number from the instrument

Delphi : function ARC_get_Mono_Serial(Mono_Enum : integer;out Serial_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Serial Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Serial_str As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Serial_Var : Returns the serial number of the instrument as a string.
Result : True if the operation was successful

ARC_get_Mono_Focallength , returns the default focal length of the instrument

Delphi : function ARC_get_Mono_Focallength(Mono_Enum : integer; out Focallength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Focallength Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Focallength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Focallength : Returns the focal length of the instrument in millimeters.
Result : True if the operation was successful

ARC_get_Mono_HalfAngle , returns the default half angle of the instrument in radians

Delphi : function ARC_get_Mono_HalfAngle(Mono_Enum : integer; out HalfAngle : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_HalfAngle Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, HalfAngle As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
HalfAngle : Returns the halfangle of the instrument in radians.
Result : True if the operation was successful

ARC_get_Mono_DetectorAngle , returns the default detector angle of the instrument in radians

Delphi : function ARC_get_Mono_DetectorAngle(Mono_Enum : integer; out DetectorAngle : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_DetectorAngle Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, DetectorAngle As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
DetectorAngle : Returns the default detector angle of the instrument in radians.
Result : True if the operation was successful

ARC_get_Mono_Double , returns if the instrument is a double monochromator

Delphi : function ARC_get_Mono_Double(Mono_Enum : integer; out Double_Present : wordbool): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Double Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Double_Present As Integer) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Double_Present : Returns true if the instrument is a double monochromator.
Result : True if the operation was successful

ARC_get_Mono_Precision , returns the nm decimal precision of the wavelength drive. Note this is independent of the wavelength step resolution whose coarseness is defined by the grating being used.

Delphi : function ARC_get_Mono_Precision(Mono_Enum : integer): integer; stdcall;
VB : Private Declare Function ARC_get_Mono_Precision Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Long

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : The number of digits after the decimal point the instrument uses. Note, the true precision is limited by the density of the grating and can be much less than the instruments wavelength precision.

ARC_get_Mono_Wavelength_nm , returns the current center wavelength of instrument in nm.

Delphi : function ARC_get_Mono_Wavelength_nm(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Wavelength_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The current wavelength of the instrument in nanometers.
Result : True if the operation was successful

ARC_set_Mono_Wavelength_nm , sets the center wavelength of the instrument in nm.

Delphi : function ARC_set_Mono_Wavelength_nm(Mono_Enum : integer; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The wavelength in nm that the instrument is to be moved to.
Result : True if the operation was successful

ARC_get_Mono_Wavelength_ang , returns the current center wavelength of instrument in Angstroms. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_ang(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Wavelength_ang Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The current wavelength of the instrument in Angstroms.
Result : True if the operation was successful

ARC_set_Mono_Wavelength_ang , sets the center wavelength of the instrument in angstroms. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_ang(Mono_Enum : integer; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_ang Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The wavelength in Angstroms that the instrument is to be moved to.
Result : True if the operation was successful

ARC_get_Mono_Wavelength_eV , returns the current center wavelength of instrument in electron Volts. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_eV(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Wavelength_eV Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The current wavelength of the instrument in Electron Volts.
Result : True if the operation was successful

ARC_set_Mono_Wavelength_eV , sets the center wavelength of the instrument in electron Volts. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_eV(Mono_Enum : integer; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_eV Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The wavelength in Electron Volts that the instrument is to be moved to.
Result : True if the operation was successful

ARC_get_Mono_Wavelength_micron , returns the current center wavelength of instrument in microns. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_micron(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Wavelength_micron Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The current wavelength of the instrument in microns.
Result : True if the operation was successful

ARC_set_Mono_Wavelength_micron , sets the center wavelength of the instrument in microns. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_micron(Mono_Enum : integer; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_micron Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The wavelength in microns that the instrument is to be moved to.
Result : True if the operation was successful

ARC_get_Mono_Wavelength_absCM , returns the current center wavelength of instrument in absolute wave number. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_absCM(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;VB : Private Declare Function ARC_get_Mono_Wavelength_absCM Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The current wavelength of the instrument in Absolute Wavenumber.
Result : True if the operation was successful

ARC_set_Mono_Wavelength_absCM , sets the center wavelength of the instrument in absolute wavenumbers. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_absCM(Mono_Enum : integer; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_absCM Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The wavelength in Absolute Wavenumber that the instrument is to be moved to.
Result : True if the operation was successful

ARC_get_Mono_Wavelength_relCM , returns the current center wavelength of instrument in relative wavenumber based upon the center line value passed to the function in nm. Converted from nm.

Delphi : function ARC_get_Mono_Wavelength_relCM(Mono_Enum : integer; Center_nm : double; out Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Wavelength_relCM Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Center_nm As Double, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Center_nm : The wavelength in nanometers that 0 relative Wavenumber is centered around.
Wavelength : The current wavelength of the instrument in Relative Wavenumber.
Result : True if the operation was successful

ARC_set_Mono_Wavelength_relCM , sets the center wavelength of the instrument in relative wave numbers centered on the wavelength passed in nm. Converted to nm and then sent the instrument.

Delphi : function ARC_set_Mono_Wavelength_relCM(Mono_Enum : integer; Center_nm : double; Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Wavelength_relCM Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Center_nm As Double, ByVal Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Center_nm : The wavelength in nanometers that 0 relative Wavenumber is centered around.
Wavelength : The wavelength in relative Wavenumber that the instrument is to be moved to.
Result : True if the operation was successful

ARC_get_Mono_Wavelength_Cutoff_nm , returns in nm the max wavelength achievable by the instrument using the current grating. Note, value might be higher than physically achievable on linear drives using the older SD-748 based controllers.

Delphi : function ARC_get_Mono_Wavelength_Cutoff_nm(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Wavelength_Cutoff_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The maximum center wavelength in nanometers for the current grating. On SpectraPro's this value equals 1400 nm * grating density / 1200 g/mm. On AM/VM products, this value is limited by the sine bar and will vary.
Result : True if the operation was successful

ARC_get_Mono_Wavelength_Min_nm , returns in nm the min wavelength achievable by the instrument using the current grating. Note, value might be lower than physically achievable on linear drives using the older SD-748 based controllers.

Delphi : function ARC_get_Mono_Wavelength_Min_nm(Mono_Enum : integer; out Wavelength : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Wavelength_Min_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Wavelength As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength : The minimum center wavelength in nanometers for the current grating. On SpectraPro's it is usually –10 nm, on linear AM/VM instruments this value will vary.
Result : True if the operation was successful

ARC_get_Mono_Turret , returns the current grating turret

Delphi : function ARC_get_Mono_Turret(Mono_Enum : integer; out Turret : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Turret Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Turret As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Turret : Current turret, One plus grating number divided by the number of gratings per turret. This value seldom exceeds three.
Result : True if the operation was successful

ARC_set_Mono_Turret , sets the current grating turret

Delphi : function ARC_set_Mono_Turret(Mono_Enum : integer; Turret : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Turret Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Turret As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.

Turret : set the current turret. This will change the grating to Turret number minus one times gratings per turret plus the one plus the current grating number minus one mod gratings per turret. The user must insert the correct turret into the instrument when using this function.

Result : True if the operation was successful

ARC_get_Mono_Turret_Gratings , returns the number of gratings per turret

Delphi : function ARC_get_Mono_Turret_Gratings(Mono_Enum : integer; out Gratings_Per_Turret : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Turret_Gratings Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Gratings_Per_Turret As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Gratings_Per_Turret : The number of gratings that can be placed on a single turret. This number can be one, two or three depending on the monochromator model.
Result : True if the operation was successful

ARC_get_Mono_Turret_Max , return the number of turrets installed

Delphi : function ARC_get_Mono_Turret_Max(Mono_Enum : integer; out Max_Turret_Number : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Turret_Max Lin “ARC_SpectraPro.dll” (ByVal Mono_Enum As Long, Max_Turret As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Max_Turret : The number of turrets installed in the monochromator. This number can be one, two, or three depending on how the instrument was configured. Some custom AM/VM monochromators with ‘snapin’ gratings have had as many as 9 turrets, this is not normal.
Result : True if the operation was successful

ARC_get_Mono_Grating , returns the current grating

Delphi : function ARC_get_Mono_Grating(Mono_Enum : integer; out Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The current grating. This assumes the correct turret has been inserted in the instrument.
Result : True if the operation was successful

ARC_set_Mono_Grating , sets the current grating

Delphi : function ARC_set_Mono_Grating(Mono_Enum : integer; Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Grating Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : Set the current grating. This assumes the correct turret is inserted in the instrument and grating selected is a valid grating. This function will change to current turret in the firmware, but needs user interaction to install the correct turret.
Result : True if the operation was successful

ARC_get_Mono_Grating_Blaze , returns the blaze of a given grating

Delphi : function ARC_get_Mono_Grating_Blaze(Mono_Enum : integer; Grating : integer; out Blaze_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating_Blaze Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, Blaze_Var As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : Which grating we are requesting the information about. Validates the request by calling ARC_get_Mono_Grating_Installed.
Blaze_Var : A variant string of the grating blaze.
Result : True if the grating requested is installed

ARC_get_Mono_Grating_Density , returns the groove density (grooves per millimeter) of a given grating

Delphi : function ARC_get_Mono_Grating_Density(Mono_Enum : integer; Grating : integer; out Groove_MM : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating_Density Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, Groove_MM As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : Which grating we are requesting the information about. Validates the request by calling ARC_get_Mono_Grating_Installed.
Groove_MM : The groove density of the grating in grooves per millimeter. For a mirror, this function will return 1200 grooves per MM.
Result : True if the grating requested is installed

ARC_get_Mono_Grating_Installed , returns if a grating is installed

Delphi : function ARC_get_Mono_Grating_Installed(Mono_Enum : integer; Grating : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating_Installed Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : Which grating we are requesting the information about.
Result : True if the grating requested is installed

ARC_get_Mono_Grating_Max , returns the max grating position installed (this is usually the number of gratings installed)

Delphi : function ARC_get_Mono_Grating_Max(Mono_Enum : integer; out Max_Grating_Number : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating_Max Lib “ARC_SpectraPro.dll” (ByVal Mono_Enum As Long , Max_Grating_Number As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Max_Grating_Number : The number of gratings installed in the monochromator. This number can range from one to nine. Some custom systems have had as many as twelve, but this is not the normal case.
Result : True if the monochromator is valid

ARC_get_Mono_Diverter_Valid , returns if a motorized diverter position is valid for an instrument

Delphi : function ARC_get_Mono_Diverter_Valid(Mono_Enum : integer; Diverter_Num : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Diverter_Valid Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Diverter_Num As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Diverter_Num : The diverter to be querried.
Mirror Position 1 … motorized entrance diverter mirror
Mirror Position 2 … motorized exit diverter mirror
Mirror Position 3 … motorized entrance diverter on a double slave unit
Mirror Position 4 … motorized exit diverter on a double slave unit
Result : True if the diverter mirror is motorized.

ARC_get_Mono_Diverter_Pos , returns the slit the mirror is pointing to

Delphi : function ARC_get_Mono_Diverter_Pos(Mono_Enum : integer; Diverter_Num : integer; out Diverter_Pos : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Diverter_Pos Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Diverter_Num As Long, Diverter_Pos As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Diverter_Num : The diverter to be queried.
Mirror Position 1 … motorized entrance diverter mirror
Mirror Position 2 … motorized exit diverter mirror
Mirror Position 3 … motorized entrance diverter on a double slave unit
Mirror Position 4 … motorized exit diverter on a double slave unit
Diverter_Pos : Port the diverter mirror is currently pointing at.
Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit
Result : True if the diverter is motorized and the operation is successful

ARC_set_Mono_Diverter_Pos , sets the port that the unit is to point to

Delphi : function ARC_set_Mono_Diverter_Pos(Mono_Enum : integer; Diverter_Num : integer; Diverter_Pos : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Diverter_Pos Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Diverter_Num As Long, ByVal Diverter_Pos As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Diverter_Num : The diverter to be modified.

Mirror Position 1 … motorized entrance diverter mirror
Mirror Position 2 … motorized exit diverter mirror
Mirror Position 3 … motorized entrance diverter on a double slave unit
Mirror Position 4 … motorized exit diverter on a double slave unit

Diverter_Pos : Port the diverter mirror is to be set to. Not all ports are valid for all diverters.

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit

Result : True if the diverter is motorized and the operation is successful

ARC_get_Mono_Diverter_Pos_Var , returns a string describing the port the mirror is pointing to

Delphi : function ARC_get_Mono_Diverter_Pos_Var(Mono_Enum : integer; Diverter_Num : integer; out Diverter_Pos : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Diverter_Pos_Var Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Diverter_Num As Long, Diverter_Pos As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Diverter_Num : The diverter to be queried.
Mirror Position 1 … motorized entrance diverter mirror
Mirror Position 2 … motorized exit diverter mirror
Mirror Position 3 … motorized entrance diverter on a double slave unit
Mirror Position 4 … motorized exit diverter on a double slave unit
Diverter_Pos : Returns as a variant a text string describing which port the diverter is pointing at.
Result : True if the diverter is motorized and the operation is successful

ARC_get_Mono_Slit_Type , returns the slit type for a given slit

Delphi : function ARC_get_Mono_Slit_Type(Mono_Enum : integer; Slit_Pos : integer; out Slit_Type : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Slit_Type Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long, Slit_Type As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Slit_Pos : The slit to be addressed.
Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit
Slit_Type : The type of slit attached
Slit Type 0 … No slit, older instruments that do will only return a slit type of zero.
Slit Type 1 … Manual Slit
Slit Type 2 … Fixed Slit Width
Slit Type 3 … Focal Plane adapter
Slit Type 4 … Continuous Motor
Slit Type 5 … Fiber Optic
Slit Type 6 … Index able Slit
Slit Type 9 … Special Continuous Motor (placed here for completeness, only one in existence, you do not have it, trust me)
Result : True if the operation was successful

ARC_get_Mono_Slit_Type_Var , returns a string descriptor of a given slit

Delphi : function ARC_get_Mono_Slit_Type_Var(Mono_Enum : integer; Slit_Pos : integer; out Slit_Type : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Slit_Type_Var Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long, Slit_Type As Variant) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Slit_Pos : The slit to be addressed.

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit

Slit_Type : **:** Returns as a variant a text string describing the slit.
Result : True if the operation was successful

ARC_get_Mono_Slit_Width , returns the slit width of a motorized slit (Types 4,6, and 9)

Delphi : function ARC_get_Mono_Slit_Width(Mono_Enum : integer; Slit_Pos : integer; out Slit_Width : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Slit_Width Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long, Slit_Width As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Slit_Pos : The slit to be addressed.

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit

Slit_Width : Returns the width of the slit if the slit is motorized.
Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful.

ARC_set_Mono_Slit_Width , sets the slit width of a motorized slit (Types 4,6, and 9)

Delphi : function ARC_set_Mono_Slit_Width(Mono_Enum : integer; Slit_Pos : integer; Slit_Width : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Slit_Width Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long, ByVal Slit_Width As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Slit_Pos : The slit to be addressed.

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit

Slit_Width : The width to which the slit is to be set to. Note valid ranges are 10 to 3000 microns in one-micron increments for normal continuous motor slits (type 4 and 9), 10 to 12000 microns in five-micron increments for large continuous motor slits (type 4) and 25,50,100,250,500,1000 microns for indexable slits (Type 6).
Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful**.**

ARC_Mono_Slit_Home , homes a motorized slit (Types 4,6, and 9)

Delphi : function ARC_Mono_Slit_Home(Mono_Enum : integer; Slit_Pos : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Slit_Home Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Slit_Pos As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Slit_Pos : The slit to be homed.

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit

Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful**.**

ARC_Mono_Slit_Name_Var , returns a text descriptor of a slit position. The same terminology is used in all Acton Research Monochromators and Spectrographs so this function is used independent of the Mono_Enum variable.

Delphi : function ARC_Mono_Slit_Name_Var(Slit_Num : integer; out Slit_Name_Var : olevariant) : wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Slit_Name_Var Lib "ARC_SpectraPro.dll" (ByVal Slit_Num As Long, Slit_Name_Var As Variant) As Integer

Slit_Pos : The slit to be addressed.
Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit
Slit_Name_Var : **:** Returns as a variant a text string describing the slit port location.
Result : True if the slit number is a valid slit

ARC_Calc_Mono_Slit_BandPass , calculates and returns the band pass for a provided slit width

Delphi : function ARC_Calc_Mono_Slit_BandPass(Mono_Enum : integer; Slit_Pos : integer; SlitWidth : double; out BandPass : double): wordbool; stdcall;
VB : Private Declare Function ARC_Calc_Mono_Slit_BandPass Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByValVal Slit_Pos as Long, ByVal SlitWidth as Double, BandPass as Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Slit_Pos : The slit to be homed.

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit

SlitWidth : The slit width that the band pass is being calculated for.
BandPass : The calculated band pass for a slit returned in nm units.
Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful**.**

ARC_get_Mono_Slit_BandPass , returns the band pass of a motorized slit (Types 4,6, and 9)

function ARC_get_Mono_Slit_BandPass(Mono_Enum : integer; Slit_Pos : integer; out BandPass : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Slit_BandPass Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByValVal Slit_Pos as Long, BandPass as Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Slit_Pos : The slit to be homed.

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit

BandPass : The calculated band pass for a slit returned in nm units.
Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful**.**

ARC_set_Mono_Slit_BandPass , sets a motorized slit width of a motorized slit (Types 4,6, and 9)

function ARC_set_Mono_Slit_BandPass(Mono_ENum : integer; Slit_Pos : integer; BandPass : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Slit_BandPass Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByValVal Slit_Pos as Long, ByVal BandPass as Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Slit_Pos : The slit to be homed.

Slit Port 1 … Side entrance slit
Slit Port 2 … Front entrance slit
Slit Port 3 … Front exit slit
Slit Port 4 … Side exit slit
Slit Port 5 … Side entrance slit on a double slave unit
Slit Port 6 … Front entrance slit on a double slave unit
Slit Port 7 … Front exit slit on a double slave unit
Slit Port 8 … Side exit slit on a double slave unit

BandPass : The band pass (in nm units) that the slit will be set to. Changes the slit width.
Result : True if the slit is motorized (Types 4,6 or 9) and the operation was successful**.**

ARC_get_Mono_Filter_Present , returns if the instrument has an integrated filter wheel **.** Note, is a new option and not available on most instruments. All filter functions call this function before proceeding.

Delphi : function ARC_get_Mono_Filter_Present(Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Filter_Present Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if an integrated filter wheel is present on the monochromator.

ARC_get_Mono_Filter_Position , returns the current filter position

Delphi : function ARC_get_Mono_Filter_Position(Mono_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Filter_Position Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Position As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Position : Returns the current filter wheel position.
Result : True if the filter wheel is present and the operation was successful

ARC_set_Mono_Filter_Position , sets the current filter position

Delphi : function ARC_set_Mono_Filter_Position(Mono_Enum : integer; Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Filter_Position Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Position As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Position : Position the filter is to be set to.
Result : True if the filter wheel is present, position is not outside of ARC_get_Mono_Filter_Min_Pos / ARC_get_Mono_Filter_Max_Pos and the operation was successful

ARC_get_Mono_Filter_Min_Pos , returns the min. filter position

Delphi : function ARC_get_Mono_Filter_Min_Pos(Mono_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Filter_Min_Pos Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Position As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Position : Returns the minimum position possible with the filter wheel.
Result : True if the filter wheel is present and the operation was successful

ARC_get_Mono_Filter_Max_Pos , returns the max. filter position

Delphi : function ARC_get_Mono_Filter_Max_Pos(Mono_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Filter_Max_Pos Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Position As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Position : Returns the maximum position possible with the filter wheel.
Result : True if the filter wheel is present and the operation was successful

ARC_Mono_Filter_Home , homes the filter wheel

Delphi : function ARC_Mono_Filter_Home(Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Filter_Home Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if the filter wheel is present, the home operation succeeded, and the operation was successful.

ARC_get_Mono_Int_Led_On , returns if the interrupter LED is on

Delphi : function ARC_get_Mono_Int_Led_On(Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Int_Led_On Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if the operation was successful and the LED is on.

Note : The LED is inside of the monochromator chamber and if turned on, should be turned off with **ARC_set_Mono_Int_LED**.

ARC_set_Mono_Int_Led , turns on and off the interrupter LED

Delphi : function ARC_set_Mono_Int_Led(Mono_Enum : integer; Led_State : wordbool): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Int_Led Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Led_State As Integer) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Led_State : Indicates if the LED is **on** or **off.**
0 – False, The LED is off
1 – True, The LED is on
Result : True if the operation was successful.

Note : The LED is inside of the monochromator chamber and if turned on, should be turned off with **ARC_set_Mono_Int_LED**.

ARC_get_Mono_Motor_Int , read the motor gear interrupter

Delphi : function ARC_get_Mono_Motor_Int(Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Motor_Int Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if the operation was successful and the interrupter is open.

0 – False, The interrupter is closed, or the LED is Off, or the operation failed
1 – True, The LED is on

ARC_get_Mono_Wheel_Int , read the wheel gear interrupter

Delphi : function ARC_get_Mono_Wheel_Int(Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Wheel_Int Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if the operation was successful and the interrupter is open.

0 – False, The interrupter is closed, or the LED is Off, or the operation failed
1 – True, The LED is on

ARC_Mono_Move_Steps , move the grating a set number of steps

Delphi : function ARC_Mono_Move_Steps(Mono_Enum : integer; Num_Steps : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Move_Steps Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Num_Steps As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Num_Steps : Number of steps to move the wavelength drive.
Result : True if the operation was successful.

ARC_get_Mono_Init_Grating , returns the initial grating (on instrument reboot)

Delphi : function ARC_get_Mono_Init_Grating(Mono_Enum : integer; out Init_Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Init_Grating Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Init_Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Init_Grating : The power-up grating number.
Result : True if the operation was successful.

ARC_set_Mono_Init_Grating , sets the initial grating (on instrument reboot)

Delphi : function ARC_set_Mono_Init_Grating(Mono_Enum : integer; Init_Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Init_Grating Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Init_Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Init_Grating : The power-up grating number.
Result : True if the operation was successful.

ARC_get_Mono_Init_Wave_nm , returns the initial wavelength (on instrument reboot)

Delphi : function ARC_get_Mono_Init_Wave_nm(Mono_Enum : integer; out Init_Wave : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Init_Wave_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Init_Wave As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Init_Wave : The power-up wavelength in nm.
Result : True if the operation was successful.

ARC_set_Mono_Init_Wave_nm , sets the initial wavelength (on instrument reboot)

Delphi : function ARC_set_Mono_Init_Wave_nm(Mono_Enum : integer; Init_Wave : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Init_Wave_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Init_Wave As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Init_Wave : The power-up wavelength in nm.
Result : True if the operation was successful.

ARC_get_Mono_Init_ScanRate_nm , returns the initial wavelength scan rate (on instrument reboot)

Delphi : function ARC_get_Mono_Init_ScanRate_nm(Mono_Enum : integer; out Init_ScanRate : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Init_ScanRate_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Init_ScanRate As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Init_ScanRate : The power-up scan rate in nm / minute.
Result : True if the operation was successful.

ARC_set_Mono_Init_ScanRate_nm , sets the initial wavelength scan rate (on instrument reboot)

Delphi : function ARC_set_Mono_Init_ScanRate_nm(Mono_Enum : integer; Init_ScanRate : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Init_ScanRate_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Init_ScanRate As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Init_ScanRate : The power-up scan rate in nm / minute.
Result : True if the operation was successful.

ARC_get_Mono_Grating_Offset , returns the offset if a grating

Delphi : function ARC_get_Mono_Grating_Offset(Mono_Enum : integer; Grating : integer; out Offset : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating_Offset Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, Offset As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The number of the grating that is to be addressed.
Offset : The grating offset. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.
Result : True if the operation was successful.

ARC_get_Mono_Grating_Gadjust , returns the GAdjust of a grating

Delphi : function ARC_get_Mono_Grating_Gadjust(Mono_Enum : integer; Grating : integer; out Gadjust : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Grating_Gadjust Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, Gadjust As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The number of the grating that is to be addressed.
GAdjust : The grating GAdjust. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.
Result : True if the operation was successful.

ARC_set_Mono_Grating_Offset , sets the grating offset

Delphi : function ARC_set_Mono_Grating_Offset(Mono_Enum : integer; Grating : integer; Offset : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Grating_Offset Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Offset As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The number of the grating that is to be addressed.
Offset : The grating offset. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.
Result : True if the operation was successful.

Note : Should only be used in conjunction with **ARC_Mono_Grating_Calc_Offset.**

ARC_set_Mono_Grating_Gadjust , sets the grating GAdjust

Delphi : function ARC_set_Mono_Grating_Gadjust(Mono_Enum : integer; Grating : integer; Gadjust : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Grating_Gadjust Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Gadjust As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The number of the grating that is to be addressed.
GAdjust : The grating GAdjust. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.
Result : True if the operation was successful.

Note : Should only be used in conjunction with **ARC_Mono_Grating_Calc_GAdjust.**

ARC_Mono_Grating_Calc_Offset , calculate a new grating offset

Delphi : function ARC_Mono_Grating_Calc_Offset(Mono_Enum : integer; Grating : integer; Wave : double; RefWave : double; out newOffset : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Grating_Calc_Offset Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Wave As Double, ByVal RefWave As Double, newOffset As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The number of the grating that is to be addressed.
Wave : The wavelength in nm on which a peak is currently falling.
RefWave : The wavelength in nm on which the peak should fall.
newOffset : The calculated grating offset. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.
Result : True if the operation was successful.

ARC_Mono_Grating_Calc_Gadjust , calculate a new grating GAdjust

Delphi : function ARC_Mono_Grating_Calc_Gadjust(Mono_Enum : integer; Grating : integer; Wave : double; RefWave : double; out newGadjust : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Grating_Calc_Gadjust Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Wave As Double, ByVal RefWave As Double, newGadjust As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The number of the grating that is to be addressed.
Wave : The wavelength in nm on which a peak is currently falling.
RefWave : The wavelength in nm on which the peak should fall.
newGAdjust : The calculated grating GAdjust. Note : this value is specific to a specific instrument grating combination and not transferable between instruments.
Result : True if the operation was successful.

ARC_Mono_Grating_UnInstall , Uninstall a grating

Delphi : function ARC_Mono_Grating_UnInstall(Mono_Enum : integer; Grating : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Grating_UnInstall Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The number of the grating that is to be uninstalled.
Result : True if the operation was successful.

ARC_Mono_Grating_Install , Install a new grating

Delphi : function ARC_Mono_Grating_Install(Mono_Enum : integer; Grating : integer; Density : Integer; Blaze : OleVariant; NMBlaze : boolean; HOLBlaze : boolean; MIRROR : boolean): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Grating_Install Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Grating As Long, ByVal Density As Long, ByVal Blaze As Variant, ByVal NMBlaze As Long, ByVal HOLBlaze As Long, ByVal MIRROR As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Grating : The number of the grating that is to be addressed.
Density : The groove density in grooves per mm of the grating.
Blaze : The Blaze String.
NMBlaze : If the grating has an nm blaze verses an micron blaze.
0 – False, Ruled grating with a micron blaze
1 – True, Ruled grating with a nm blaze
HOLBlaze : If the grating has a Holographic Blaze.
0 – False, Is not a Holographic Grating
1 – True, Is a Holographic Grating
MIRROR : If the grating position is a mirror.
0 – False, Not a mirror
1 – True, Is a mirror
Result : True if the operation was successful.

ARC_Mono_Reset , Reboot an instrument

Delphi : function ARC_Mono_Reset(Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Reset Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if the operation was successful and the instrument was reset to the power-on state.

ARC_Mono_Restore_Factory_Settings , restore the instrument factory settings (not available of the oldest instruments)

Delphi : function ARC_Mono_Restore_Factory_Settings(Mono_Enum : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Restore_Factory_Settings Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if the operation was successful.

Note : It is highly recommended that this function not be invoked.

ARC_Mono_Save_Factory_Settings, this definition is reserved and should never be invoked

Delphi : reserved
VB : reserved

Reserved : reserved

ARC_get_Mono_Scan_Rate_nm_min , return the current wavelength scan rate

Delphi : function ARC_get_Mono_Scan_Rate_nm_min (Mono_Enum : integer; out Scan_Rate : double): wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_Scan_Rate_nm_min Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Scan_Rate As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Scan_Rate : Scan rate in nm per minute.
Result : True if the operation was successful.

ARC_set_Mono_Scan_Rate_nm_min , set the wavelength scan rate

Delphi : function ARC_set_Mono_Scan_Rate_nm_min (Mono_Enum : integer; Scan_Rate : double): wordbool; stdcall;
VB : Private Declare Function ARC_set_Mono_Scan_Rate_nm_min Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Scan_Rate As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Scan_Rate : The rate we are to scan the wavelength in nm/minute.
Result : True if the operation was successful.

ARC_Mono_Start_Scan_To_nm , start a wavelength scan

Delphi : function ARC_Mono_Start_Scan_To_nm (Mono_Enum : integer; Wavelength_nm : double): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Start_Scan_To_nm Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, ByVal Wavelength_nm As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Wavelength_nm : Wavelength in nm we are to scan to. Note should not be lower than the current wavelength and should not exceed the current grating wavelength limit.
Result : True if the operation was successful.

ARC_Mono_Scan_Done , check if a scan has completed

Delphi : function ARC_Mono_Scan_Done (Mono_Enum : integer; out Done_Moving : boolean;out Current_Wavelength_nm : double): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Scan_Done Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long, Done_Moving As Long, Current_Wavelength_nm As Double) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Done_Moving : Indicate if we are done moving (the scan ended or the grating wavelength limit was reached)
0 – False, we are still moving
1 – True, we are done moving
Current_Wavelength_nm : The wavelength in nm of the grating when this command was issued.
Result : True if the operation was successful.

Note : Once a scan or a jog is completed a **ARC_Mono_Stop_Jog** must be issued once before an accurate final wavelength can be read with **ARC_get_Mono_Wavelength_nm.**

ARC_Mono_Start_Jog , start jogging the wavelength of an instrument

Delphi : function ARC_Mono_Start_Jog (Mono_Enum : integer; Jog_MaxRate : boolean; JogUp : boolean): wordbool; stdcall;
VB : Use not recommended in the VB enviroment

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Jog_MaxRate : If we jog at the max scan rate or current scan rate.
0 – Current Scan Rate
1 – Max Scan Rate
JogUp : If we jog forward (increasing nm) or backwards (decreasing nm)
0 – Jog Backwards (decreasing nm)
1 – Jog Forwards (increasing nm)
Result : True if the operation was successful.

ARC_Mono_Stop_Jog , end a wavelength jog

Delphi : function ARC_Mono_Stop_Jog (Mono_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Mono_Stop_Jog Lib "ARC_SpectraPro.dll" (ByVal Mono_Enum As Long) As Integer

Mono_Enum : The handle defined in ARC_Open_Mono by which the monochromator is to be addressed.
Result : True if the operation was successful.

ARC_Open_Filter , function to open a Filter Wheel

Delphi : function ARC_Open_Filter(Enum_Num : integer; out Filter_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Open_Filter Lib "ARC_FilterWheel.dll" (ByVal Enum_Num As Long, Filter_Enum As Long) As Integer

Enum_Num : Number in the enumeration list to open.
Filter_Enum : The handle by which to address the Filter Wheel in all future calls. Can be validated with ARC_Valid_Filter_Enum.
Result : True, if it successfully opened the device.

ARC_Open_Filter_Port , function to open a Filter Wheel on a specific COM Port (ie. COM1, COM2, ect…)

Delphi : function ARC_Open_Filter_Port(Com_Num : integer; out Filter_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Open_Filter_Port Lib "ARC_FilterWheel.dll" (ByVal Com_Num As Long, Filter_Enum As Long) As Integer

Com_Num : Com Port (COM1,COM2,ect.) to open.
Filter_Enum : The handle by which to address the Filter Wheel in all future calls. Can be validated with ARC_Valid_Filter_Enum.
Result : True, if it successfully opened the device.

ARC_Valid_Filter_Enum , function returns if a Filter Wheel enumeration is both valid and the instrument is open. All functions call this function to verify that the requested action is valid.

Delphi : function ARC_Valid_Filter_Enum(Filter_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Valid_Filter_Enum Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Result : True if Filter_Enum is a valid Filter Wheel that is open.

ARC_get_Filter_preOpen_Model , returns the model string of an instrument not yet opened. Note, ARC_Search_For_Filter needs to be called prior to this call. In the case of multiple Filter Wheel/Spectrographs attached, it allows the user to sort, which instruments are to be opened before opening them.

Delphi : function ARC_get_Filter_preOpen_Model(Enum_Num : integer; out Model_Var : olevariant) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_preOpen_Model Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long, Model_Var As Variant) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Model_Var : The model string of the unopened Filter Wheel.
Result : True if the operation was successful

ARC_Send_CMD_To_Filter , this is an internal function that provides direct communications with the instrument. Usage of this function should be avoided.

Delphi : function ARC_Send_CMD_To_Filter(Filter_Enum : integer; SendCMD : olevariant; out RCV_Var : olevariant; ms_Timeout : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_Send_CMD_To_Filter Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long, ByVal SendCMD As Variant, RCV_Var As Variant, ByVal ms_Timeout As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
SendCMD : The text command to be sent to the instrument. Note, line termination is handled by the DLL.
RCV_Var : The reply from the instrument to the given command.
Ms_Timeout : How long the DLL should wait in milliseconds for an acknowledgement before returning
Result : True if the instrument did not reject the command or timeout.

ARC_get_Filter_Model , returns the model string from the instrument

Delphi : function ARC_get_Filter_Model(Filter_Enum : integer;out Model_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Model Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long, Model_str As Variant) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Model_Var : Returns the model string of the instrument.
Result : True if the operation was successful

ARC_get_Filter_Serial , returns the serial number from the instrument. Note, older filter controllers did not electronically store the serial number. In the case of the older controllers Serial_Var will not contain any text.

Delphi : function ARC_get_Filter_Serial(Filter_Enum : integer;out Serial_Var : olevariant): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Serial Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long, Serial_str As Variant) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Serial_Var : Returns the serial number of the instrument as a string.
Result : True if the operation was successful

ARC_get_Filter_Present , returns if the instrument has a filter wheel **.** Note if ARC_Valid_Filter_Enum returns true, this function will also return true.

Delphi : function ARC_get_Filter_Present(Filter_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Present Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Result : True if an integrated filter wheel is present on the Filter Wheel.

ARC_get_Filter_Position , returns the current filter position

Delphi : function ARC_get_Filter_Position(Filter_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Position Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long, Position As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Position : Returns the current filter wheel position.
Result : True if the filter wheel is present and the operation was successful

ARC_set_Filter_Position , sets the current filter position

Delphi : function ARC_set_Filter_Position(Filter_Enum : integer; Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_set_Filter_Position Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long, ByVal Position As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Position : Position the filter is to be set to.
Result : True if the filter wheel is present, position is not outside of ARC_get_Filter_Min_Pos / ARC_get_Filter_Max_Pos and the operation was successful

ARC_get_Filter_Min_Pos , returns the min. filter position

Delphi : function ARC_get_Filter_Min_Pos(Filter_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Min_Pos Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long, Position As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Position : Returns the minimum position possible with the filter wheel.
Result : True if the filter wheel is present and the operation was successful

ARC_get_Filter_Max_Pos , returns the max. filter position

Delphi : function ARC_get_Filter_Max_Pos(Filter_Enum : integer; out Position : integer): wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_Max_Pos Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long, Position As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Position : Returns the maximum position possible with the filter wheel.
Result : True if the filter wheel is present and the operation was successful

ARC_Filter_Home , homes the filter wheel

Delphi : function ARC_Filter_Home(Filter_Enum : integer): wordbool; stdcall;
VB : Private Declare Function ARC_Filter_Home Lib "ARC_FilterWheel.dll" (ByVal Filter_Enum As Long) As Integer

Filter_Enum : The handle defined in ARC_Open_Filter by which the Filter Wheel is to be addressed.
Result : True if the filter wheel is present, the home operation succeeded, and the operation was successful.

ARC_get_Mono_preOpen_Model_int32 , returns the model type of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call.

Delphi : function ARC_get_Mono_preOpen_Model_Int32(Enum_Num : integer; out Model_Int32 : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_preOpen_Model_int32(ByVal Enum_Num As Long , Model_Int32 As Long) As Integer

Enum_Num : The index as defined by ARC_Search_For_Inst for the instrument addressed.
Model_Int32 : The model type as a integer

Model Type –1 ... Unknown, not supported
Model Type 0 ... Unknown, not supported
Model Type 1 ... GIMS (reserved for, none currently supported by dll)
Model Type 2 ... SP-150 (superceded by SP-2-150 (SPv6x) in 2003)
Model Type 3 ... SPx00i (generic gen2 board) (superceded by SPv6x in 2003)
Model Type 4 ... SPx004i (generic SD-2) (superceded by SD-3 in 2003)
Model Type 5 ... SP-300i (superceded by SP-2-300i (SPv6x) in 2003)
Model Type 6 ... SP-500i (superceded by SP-2-500i (SPv6x) in 2003)
Model Type 7 ... SP-750i (superceded by SP-2-750i (SPv6x) in 2003)
Model Type 8 ... INS-300 (serial number is < 42) (superceded by SPv6x based INS-300 in 2003)
Model Type 9 ... VM-502i (VM-502 with SD-2) (superceded by SD-3 based VM-502 in 2003)
Model Type 10 ... VM-504i (VM-504 with SD-2) (superceded by SD-3 based VM-504 in 2003)
Model Type 11 ... VM-503 (VM-503 with SD-2) (superceded by SD-3 based VM-503 in 2003)
Model Type 12 ... VM-505 (VM-505 with SD-2) (superceded by SD-3 based VM-505 in 2003)
Model Type 13 ... VM-506 (VM-506 with SD-2) (superceded by SD-3 based VM-506 in 2003)
Model Type 14 ... VM-507 (VM-507 with SD-2) (superceded by SD-3 based VM-507 in 2003)
Model Type 15 ... VM-510 (VM-510 with SD-2) (superceded by SD-3 based VM-510 in 2003)
Model Type 16 ... VM-511 (VM-511 with SD-2) (superceded by SD-3 based VM-511 in 2003)
Model Type 17 ... VM-512 (VM-512 with SD-2) (superceded by SD-3 based VM-512 in 2003)
Model Type 18 ... VM-521 (VM-521 with SD-2) (superceded by SD-3 based VM-521 in 2003)
Model Type 19 ... VM-523 (VM-523 with SD-2) (superceded by SD-3 based VM-523 in 2003)
Model Type 20 ... SP-275 (superceded by SP-300i in 1996)
Model Type 21 ... SP-500 (SP-500 with SD-748) (superceded by SP-500i in 1996)
Model Type 22 ... SP-750 (SP-750 with SD-748) (superceded by SP-750i in 1998)
Model Type 23 ... VM-502 (VM-502 with SD-748) (superceded by VM-502i in 1999)
Model Type 24 ... VM-504 (VM-504 with SD-748) (superceded by VM-504i in 1999)
Model Type 25 ... SD-748 (superceded by SD-2 in 1999)
Model Type 26 ... INS-150
Model Type 27 ... SPv6x (All SpectraPro's produced since 2003,
Includes: SP-2-150, SP-2-300i, SP-2-500i, SP-2-750i, SD-3,
InSight, MS-2-150i, MS-2-300i, INS-300 serial numbers > 42)
Model Type 28 ... NCL (superceded by SpectraHub in 2003)
Model Type 29 ... Filter Controller
Model Type 30 ... Reserved for Sample Chambers
Model Type 31 ... Reserved (custom, not supported by dll)
Model Type 32 ... Reserved (custom, not supported by dll)
Model Type 33 ... SpectraHub

Result : True if the operation was successful

ARC_get_Filter_preOpen_Model_int32, returns the model type of an instrument not yet opened. Note, ARC_Search_For_Inst needs to be called prior to this call.

Delphi : function ARC_get_Filter_preOpen_Model_int32 (Enum_Num : integer; out Model_Int32 : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_preOpen_Model_int32 (ByVal Enum_Num As Long , Model_Int32 As Long) As Integer

Enum_Num : The index as defined by ARC_Search_For_Inst for the instrument addressed.
Model_Int32 : The model type as a integer

Model Type –1 ... Unknown, not supported
Model Type 0 ... Unknown, not supported
Model Type 1 ... GIMS (reserved for, none currently supported by dll)
Model Type 2 ... SP-150 (superceded by SP-2-150 (SPv6x) in 2003)
Model Type 3 ... SPx00i (generic gen2 board) (superceded by SPv6x in 2003)
Model Type 4 ... SPx004i (generic SD-2) (superceded by SD-3 in 2003)
Model Type 5 ... SP-300i (superceded by SP-2-300i (SPv6x) in 2003)
Model Type 6 ... SP-500i (superceded by SP-2-500i (SPv6x) in 2003)
Model Type 7 ... SP-750i (superceded by SP-2-750i (SPv6x) in 2003)
Model Type 8 ... INS-300 (serial number is < 42) (superceded by SPv6x based INS-300 in 2003)
Model Type 9 ... VM-502i (VM-502 with SD-2) (superceded by SD-3 based VM-502 in 2003)
Model Type 10 ... VM-504i (VM-504 with SD-2) (superceded by SD-3 based VM-504 in 2003)
Model Type 11 ... VM-503 (VM-503 with SD-2) (superceded by SD-3 based VM-503 in 2003)
Model Type 12 ... VM-505 (VM-505 with SD-2) (superceded by SD-3 based VM-505 in 2003)
Model Type 13 ... VM-506 (VM-506 with SD-2) (superceded by SD-3 based VM-506 in 2003)
Model Type 14 ... VM-507 (VM-507 with SD-2) (superceded by SD-3 based VM-507 in 2003)
Model Type 15 ... VM-510 (VM-510 with SD-2) (superceded by SD-3 based VM-510 in 2003)
Model Type 16 ... VM-511 (VM-511 with SD-2) (superceded by SD-3 based VM-511 in 2003)
Model Type 17 ... VM-512 (VM-512 with SD-2) (superceded by SD-3 based VM-512 in 2003)
Model Type 18 ... VM-521 (VM-521 with SD-2) (superceded by SD-3 based VM-521 in 2003)
Model Type 19 ... VM-523 (VM-523 with SD-2) (superceded by SD-3 based VM-523 in 2003)
Model Type 20 ... SP-275 (superceded by SP-300i in 1996)
Model Type 21 ... SP-500 (SP-500 with SD-748) (superceded by SP-500i in 1996)
Model Type 22 ... SP-750 (SP-750 with SD-748) (superceded by SP-750i in 1998)
Model Type 23 ... VM-502 (VM-502 with SD-748) (superceded by VM-502i in 1999)
Model Type 24 ... VM-504 (VM-504 with SD-748) (superceded by VM-504i in 1999)
Model Type 25 ... SD-748 (superceded by SD-2 in 1999)
Model Type 26 ... INS-150
Model Type 27 ... SPv6x (All SpectraPro's produced since 2003,
Includes: SP-2-150, SP-2-300i, SP-2-500i, SP-2-750i, SD-3,
InSight, MS-2-150i, MS-2-300i, INS-300 serial numbers > 42)
Model Type 28 ... NCL (superceded by SpectraHub in 2003)
Model Type 29 ... Filter Controller
Model Type 30 ... Reserved for Sample Chambers
Model Type 31 ... Reserved (custom, not supported by dll)
Model Type 32 ... Reserved (custom, not supported by dll)
Model Type 33 ... SpectraHub

Result : True if the operation was successful

ARC_get_Filter_preOpen_Serial_int32 , returns the serial number (as an integer) of a filterwheel not yet open.

Delphi : function ARC_get_Filter_preOpen_Serial_int32(Filter_Num : integer; out Serial_Int32 : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Filter_preOpen_Serial_int32 Lib “ARC_Instrument.dll” (ByVal Filter_Num As Long , Serial_Int32 As Long) As Integer

Filter_Num : The index as defined by ARC_Search_For_Inst for the instrument addressed.
Serial_Int32 : Returns the serial number as an integer.
Result : True if Filter_Num refers to a valid filterwheel.

ARC_get_Mono_preOpen_Serial_int32 , returns the serial number (as an integer) of a monochromator not yet open.

Delphi : function ARC_get_Mono_preOpen_Serial_int32(Mono_Num : integer; out Serial_Int32 : integer) : wordbool; stdcall;
VB : Private Declare Function ARC_get_Mono_preOpen_Serial_int32 Lib “ARC_Instrument.dll” (ByVal Mono_Num As Long , Serial_Int32 As Long) As Integer

Mono_Num : The index as defined by ARC_Search_For_Inst for the instrument addressed.
Serial_Int32 : Returns the serial number as an integer.
Result : True if Mono_Num refers to a valid monochromator.